PROGRAMAÇÃO FUNCIONAL

TIPOS ALGÉBRICOS DE DADOS E POLIMORFISMO

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

OBJETIVOS DA AULA

- Tipos de **dados** customizados utilizando a palavra-chave **data**.
- **Aplicar casamento** de padrões sobre **estruturas** de dados **definidas** pelo **usuário**.
- **Introduzir** estruturas **recursivas** e **parametrização** de **tipos** (Polimorfismo)

A NECESSIDADE DE TIPOS PRÓPRIOS

Na **modelagem** de sistemas, utilizar tipos **primitivos** (como Int ou String) para representar conceitos de **domínio restrito** frequentemente resulta em **inconsistências**. Se modelarmos os **dias da semana** com números de 1 a 7, o compilador não nos impedirá de passar o valor 8, podendo gerar falhas.

A NECESSIDADE DE TIPOS PRÓPRIOS

Como **garantir**, estaticamente, que uma variável contenha **apenas** valores estritamente **válidos** para o contexto?

TIPOS BÁSICOS E ENUMERAÇÕES

A forma mais **elementar** de um **Tipo de Dado Abstrato** (TAD) é a **enumeração**.
Cada **alternativa** é um **construtor** de dados **sem parâmetros** associados.
O **casamento de padrão** é utilizado para **desestruturar** e **interagir** com esses valores.

TIPOS BÁSICOS E ENUMERAÇÕES

```
1.-- Definicao de um tipo enumerado
2.data DiaSemana = Domingo | Segunda | Terca
    | Quarta | Quinta | Sexta | Sabado
3.
4.-- Casamento de padrao sobre a enumeracao
5.ehFimDeSemana :: DiaSemana -> Bool
6.ehFimDeSemana Domingo = True
7.ehFimDeSemana Sabado  = True
8.ehFimDeSemana _       = False
```

O AGRUPAMENTO LÓGICO DE INFORMAÇÕES

Frequentemente, propriedades **isoladas** não possuem **significado** semântico quando **separadas**. Um **ponto** em um **plano cartesiano** não é apenas um valor **Double**, mas sim um **par** inseparável de **coordenadas** (X, Y).

O AGRUPAMENTO LÓGICO DE INFORMAÇÕES

Passar essas variáveis **separadamente** para **funções** quebra a **coesão** do código e **dificulta** a **manutenção**. Como **encapsular** múltiplos **valores** em uma única **entidade** formal?

CONSTRUTORES DE DADOS E PONTOS NO PLANO CARTESIANO

Construtores podem **encapsular** dados, formando o que chamamos de Tipos de Produto.
Além de **estruturarem** os dados, **construtores** em Haskell operam **semanticamente** como **funções** de **ordem superior** que recebem **argumentos** e retornam **instâncias** do tipo algébrico.

CONSTRUTORES DE DADOS E PONTOS NO PLANO CARTESIANO

```
1.-- O tipo Ponto2D possui um unico
2.-- construtor Ponto que encapsula duas
3.-- coordenadas de ponto flutuante.
4.
5.data Ponto2D = Ponto Double Double
```

CONSTRUTORES DE DADOS E PONTOS NO PLANO CARTESIANO

```
1.-- O construtor 'Ponto' possui a seguinte
  assinatura implicita:
2.-- Ponto :: Double -> Double -> Ponto2D
3.
4.-- Acesso aos dados via pattern matching
5.distanciaOrigem :: Ponto2D -> Double
6.distanciaOrigem (Ponto x y) = sqrt (x^2 + y^2)
```

VARIAÇÃO ESTRUTURAL

Como **representar** uma **forma geométrica** que pode ser um **círculo** (exigindo apenas um ponto central e um raio) ou um **retângulo** (exigindo dois pontos extremos), mantendo a **tipagem** estática e sem recorrer a **classes** e **herança**?

VARIAÇÃO ESTRUTURAL

Em linguagens funcionais, um **único** tipo é capaz de assumir **diferentes morfologias** de maneira **segura**.

FORMAS GEOMÉTRICAS (MÚLTIPLOS CONSTRUTORES)

É possível **combinar Tipos de Soma** e de **Produto**.
Um **único** tipo de dado pode apresentar **múltiplos construtores**, cada um com **diferentes** tipos e **quantidades** de parâmetros.

FORMAS GEOMÉTRICAS (MÚLTIPLOS CONSTRUTORES)

```
1.-- Definicao de Formas Geometricas
2.data Forma = Circulo Ponto2D Double
3.           | Retangulo Ponto2D Ponto2D
```

FORMAS GEOMÉTRICAS (MÚLTIPLOS CONSTRUTORES)

```
1.-- Calculo da area casando padrão
2.
3.calculaArea :: Forma -> Double
4.calculaArea (Circulo _ raio) = pi * (raio ^ 2)
5.calculaArea (Retangulo (Ponto x1 y1) (Ponto x2 y2)) =
6.    abs (x2 - x1) * abs (y2 - y1)
```

ESTRUTURAS DE TAMANHO ARBITRÁRIO

Os TADs, Tipos Abstratos de Dados, vistos até agora possuem **tamanho fixo** na memória. **Contudo**, aplicações **reais** requerem **coleções de dados** cuja magnitude **cresce dinamicamente**.

ESTRUTURAS DE TAMANHO ARBITRÁRIO

Se **não** podemos **modificar** variáveis nem **alocar** vetores imperativamente, como o **paradigma funcional** modela estruturas de dados como **listas** e **árvores**?

TIPOS RECURSIVOS

Tipos algébricos permitem **autorreferência**, o que possibilita a construção de estruturas de dados **dinâmicas**, como **listas** encadeadas e **árvores**.

TIPOS RECURSIVOS

```
-- Definicao de uma lista encadeada de
inteiros
-- Um No contem um inteiro e uma referencia
ao resto da lista
data ListaInt = Vazia
              | No Int ListaInt

-- Criacao de uma lista: [1, 2, 3]
minhaLista :: ListaInt
minhaLista = No 1 (No 2 (No 3 Vazia))
```

TIPOS RECURSIVOS

Na **prática**, uma **lista recursiva** em Haskell é **estritamente** uma **expansão** de **construtores aninhados** da direita para a esquerda. Cada uso do **construtor No empacota** um **elemento** e a estrutura **seguinte**.

NAVEGAÇÃO EM ESTRUTURAS DINÂMICAS

Diferente de **vetores** em linguagens iterativas, **não** existe "índice 0" ou "índice 1" em uma **estrutura recursiva** como a que acabamos de criar. Como **percorrer**, **buscar** ou **manipular** uma **coleção** de **construtores aninhados** se não temos acesso **posicional** e não dispomos de **laços**?

BUSCA SEQUENCIAL LISTAS

A **iteração** sobre estruturas **recursivas** é realizada **naturalmente** por meio de **indução estrutural** (recursividade).
A função **processa** a **base** e o **passo indutivo avaliando** se o elemento procurado coincide com a cabeça da estrutura em determinado nó.

BUSCA SEQUENCIAL LISTAS

```
-- Algoritmo de busca linear

buscaSequencial :: Int -> ListaInt -> Bool
buscaSequencial _ Vazia = False
buscaSequencial alvo (No valor resto)
    | alvo == valor = True
    | otherwise     = buscaSequencial alvo resto
```

A AUSÊNCIA DE VALOR E A GENERALIZAÇÃO

Nossa função de **busca** atual retorna um **booleano** (True/False), o que é **útil**, mas **insuficiente** caso precisássemos **recuperar** um **dado** complexo. O que fazer quando uma **função** de recuperação **não encontra** o alvo?

A AUSÊNCIA DE VALOR E A GENERALIZAÇÃO

Retornar 0 ou null **quebra** a **segurança** de tipos.
Adicionalmente, por que escreveríamos uma **ListaInt**, uma **ListaString** e uma **ListaDouble** se a lógica de **encadeamento** é **idêntica**?

TIPOS POLIMÓRFICOS E O TIPO MAYBE

Para garantir o **reuso**, tipos algébricos podem receber **variáveis** de tipo como **parâmetros**, caracterizando o **polimorfismo paramétrico**.

TIPOS POLIMÓRFICOS E O TIPO MAYBE

O tipo **Maybe**, presente no **Prelude**, **encapsula** cálculos que podem falhar, **substituindo** a necessidade de **exceções** em tempo de execução.

TIPOS POLIMÓRFICOS E O TIPO MAYBE

```
-- Definicao nativa de Maybe
-- (Tipo polimorfico parametrizado em 'a')
-- data Maybe a = Nothing | Just a
```

TIPOS POLIMÓRFICOS E O TIPO MAYBE

1. -- **Aplicacao**: Divisao segura que previne erros de divisao por zero
2.
3. divisaoSegura :: Double -> Double -> Maybe Double
4. divisaoSegura _ 0 = Nothing
5. divisaoSegura x y = Just (x / y)

EXERCÍCIOS

Desafio 1: Enumerações Estacionais
Implemente um tipo **enumerado** "Estacao" representando as quatro estações do ano (Primavera, Verao, Outono, Inverno).
Em seguida, crie uma **função** "proximaEstacao :: Estacao -> Estacao" que utilize o **casamento de padrão** para retornar a estação cronologicamente **subsequente** àquela fornecida como parâmetro.

EXERCÍCIOS

Desafio 2: Manipulação de Construtores
Reaproveitando o tipo Ponto2D, projete uma função "transladaPonto :: Ponto2D -> Double -> Double -> Ponto2D". A função deve **extrair** as **coordenadas** reais de um **ponto fornecido**, **adicionar** deslocamentos **dx** e **dy** às variáveis correspondentes e **aplicar** o construtor como função para **instanciar** e **retornar** o **novo** estado do **ponto**.

EXERCÍCIOS

Desafio 3: Tipos de Soma e Operações Financeiras
Defina um **TAD** chamado "MetodoPagamento" que possua **três construtores**: **Dinheiro** (sem parâmetros adicionais), **Pix** (contendo uma String com a chave) e **CartaoCredito** (contendo uma String com a bandeira e uma String com o número do cartão).
Por fim, elabore a **função "**processaPagamento :: MetodoPagamento -> String" que retorna **mensagens** textuais **específicas** baseadas no **construtor** extraído no padrão.

EXERCÍCIOS

Desafio 4: Indução em Estruturas Recursivas
Baseando-se no tipo **ListaInt** apresentado em aula, projete uma **função** recursiva "somaElementos :: ListaInt -> Int". A função deverá **avaliar** cada **nó** da lista encadeada customizada, **acumulando** a **soma aritmética** de **todos** os **inteiros** nela contidos.

EXERCÍCIOS

Desafio 5: Segurança com o Tipo Maybe
Implemente uma **variação** segura do algoritmo "busca Sequencial" apresentado em aula. **Crie** a **função** "buscaSegura :: Int -> ListaInt -> Maybe Int". A função deve **percorrer** a lista e retornar Just x caso o elemento x seja localizado, ou Nothing caso a busca finalize sem encontrar correspondência.

DÚVIDAS???